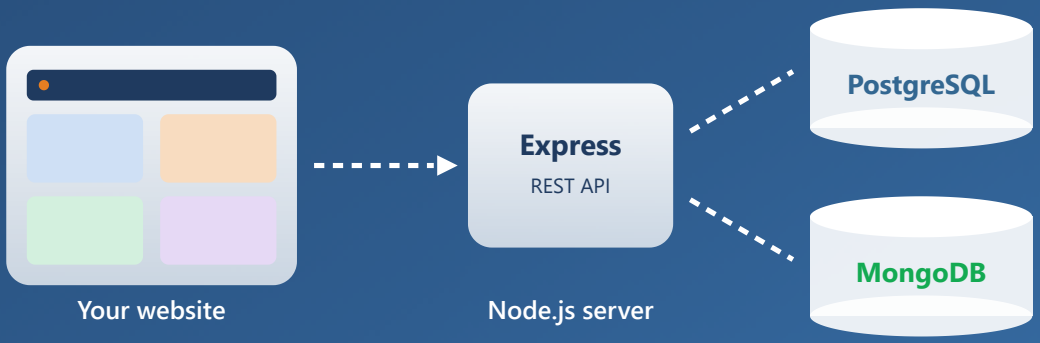
BIG DATA · STUDENT PROJECT GUIDE

Build Your Own Database + Website in 3 Weeks

PostgreSQL (pgAdmin) · MongoDB (Compass) · Node.js + Express · free cloud hosting. Step-by-step, week by week, with tested source code you can run in the lab today.



3 weeks
15 lab/home days, ~12 h per week

11 project ideas
startup-ready, with entities

100 points
weekly checklists + final rubric

How to use this guide

In this project you will design and build a **real data-driven website**: your own database (PostgreSQL for business data, MongoDB for large, flexible data such as reviews and click events), a Node.js web server that reads and writes that data, and a website that anyone can open from a link. You finish in **3 weeks**.

The reference project: "SavdoShop"
Every step uses one complete, working example: a small online store. All its code is in the folder `BigData_Starter_Code` that your teacher gives you. It has been **tested end-to-end** (schema, 500 000-row queries, MongoDB scripts and every API route). You copy it, run it, then **change it into your own project** (restaurant, bank, pharmacy …).

How each week works
Each week has: a day-by-day plan → numbered steps with screenshots → code to type and run → "Your turn" tasks for *your* project → a deliverable → a grading checklist. Tick the boxes as you go. Stuck? Read **Chapter 7 – Troubleshooting** first, it covers the most common lab errors.

Golden rules

1. Run the reference code **first**, make it work, **then** change it. Never write everything from zero.
2. Change one thing at a time and test it immediately (refresh the page, rerun the query).
3. Commit to GitHub at the end of **every** lab session. A lost USB stick is not an excuse.
4. Never put passwords in your code or on GitHub – they go into a file called `.env`.

Contents

1. Course plan, workload & tools – timeline, architecture
2. Choose your project – 11 startup ideas with entities
3. Week 1 – Design & build the database – pgAdmin, SQL, MongoDB
4. Week 2 – Build the website & API – Node.js, Express
5. Week 3 – Deploy, test & present – Supabase, Atlas, Render
6. Report & presentation templates
7. Troubleshooting – common errors
8. Grading rubric (100 points)
9. Appendix A – SQL & MongoDB cheat sheets
10. Appendix B – Full server source code

What you will be able to do after 3 weeks

- [ ] Design a normalised relational schema (tables, primary/foreign keys, constraints) and draw its ER diagram.
- [ ] Write SQL for CRUD, multi-table JOINs, GROUP BY/HAVING, views, window functions and transactions.
- [ ] Generate and analyse 500 000+ rows, and prove with `EXPLAIN ANALYZE` how an index makes a query ~10× faster.
- [ ] Model document data in MongoDB and write aggregation pipelines (`$match`, `$group`, `$unwind`, `$sort`).
- [ ] Build a REST API in Node.js/Express that uses **both** databases safely (parameterised queries, transactions).
- [ ] Deploy a database-backed website for free and present it like a startup founder.

1 · Course plan, workload & tools

1.1 The 3-week timeline

WEEK 1 · Database	WEEK 2 · Website + API	WEEK 3 · Deploy + Demo
• Install check, pick idea	• Node.js project, .env	• Cloud DBs (Supabase, Atlas)
• ER diagram, schema SQL	• Connect pg + MongoDB	• Deploy on Render (free)
• Sample data (30+ rows)	• REST API (8+ routes)	• Test + fix + polish
• 20 SQL + 12 Mongo queries	• HTML pages + fetch()	• Report (8–12 pages)
• 500k rows + index test	• Order/booking transaction	• 7-minute live demo
● Deliverable 1: DB + queries	● Deliverable 2: working local site	● Deliverable 3: public URL + report

Figure 1.1 – Three weeks, three deliverables. Each deliverable is graded with the checklist at the end of its week.

1.2 Weekly workload

Week	In the lab (with teacher)	At home / self-study	Total	Deliverable (due end of week)	Points
1	2 sessions × 2 h: tools, pgAdmin, SQL, Compass	~8 h: ER diagram, your schema, sample data, query practice	12 h	Database scripts (.sql, .js) + ER diagram + query screenshots	30
2	2 sessions × 2 h: Node/Express, API, front-end	~8 h: your API routes, pages, transaction, GitHub	12 h	Website running on `localhost:3000` + GitHub repo	30
3	2 sessions × 2 h: deployment, testing, rehearsal	~8 h: deploy, report, slides, video backup	12 h	Public URL + report + presentation	40

Teams: work alone or in pairs (pairs must build 2 extra features – see each week). Every student must be able to explain every line of their project during the demo.

1.3 Your free toolkit

Purpose	Tool (all free)	Where	Notes
Relational database	**PostgreSQL 16/17** + **pgAdmin 4**	Installed in lab	Home: postgresql.org/download (installer includes pgAdmin)
Document database	**MongoDB Community** + **Compass**	Installed in lab	Home: mongodb.com/try/download/community
Web server runtime	**Node.js 20 or 22 LTS**	nodejs.org	Check with `node -v`
Code editor	**VS Code** + extensions “MongoDB for VS Code”, “Thunder Client”	code.visualstudio.com	Thunder Client = test API without writing a page
Version control	**Git** + **GitHub** account	git-scm.com, github.com	Use your university e-mail for the GitHub Student Pack

Purpose	Tool (all free)	Where	Notes
Cloud PostgreSQL	**Supabase** free tier (or **Neon** free tier)	supabase.com	500 MB free; project pauses after 7 days without use
Cloud MongoDB	**MongoDB Atlas M0**	mongodb.com/atlas	512 MB free forever
Hosting the website	**Render** free web service	render.com	Sleeps after 15 min idle; first visit takes ~50 s
Instant demo link	**Cloudflare quick tunnel** (`cloudflared`)	developers.cloudflare.com	Public https link to your laptop, no account needed

1.4 Architecture – how the pieces talk

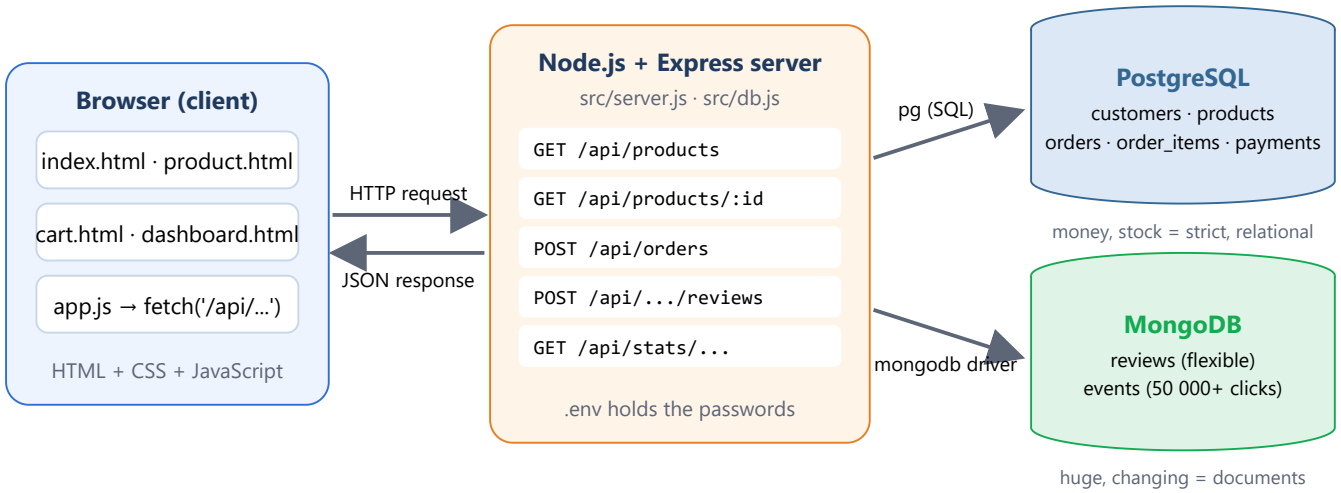


Figure 1.2 – Polyglot persistence: transactional data in PostgreSQL, high-volume/flexible data in MongoDB, one Express API in front of both.

Why two databases? (the "Big Data" idea)

Real companies such as Uzum Market or Yandex Go keep **money and stock** in a relational database because they need transactions and strict rules (a balance must never be negative). They keep **click streams, logs and reviews** in a document/NoSQL store because that data is huge, arrives fast and changes shape. Your project copies this design on a small scale.

2 · Choose your project

Pick **one** idea (or propose your own – the teacher must approve it by the end of Day 2). Each card lists the minimum features and the database entities you will need. Orange = classic business; green = unique ideas built for the Uzbekistan / Central Asia market. Every idea can grow into a startup after the course.

1. Restaurant ordering food-tech

Menu with categories, table/QR or delivery orders, kitchen status board, popular dishes report.

`SQL: restaurants, menu_categories, dishes, customers, orders, order_items, tables, payments`
`Mongo: reviews, kitchen_events`

2. Bank / digital wallet fintech

Accounts, cards, transfers between accounts (transaction!), statements, monthly spending by category.

`SQL: clients, accounts, cards, transactions, transfers, merchants, categories`
`Mongo: login_events, fraud_alerts`

3. Jewelry store luxury retail

Products by metal/stone/weight, live gold price × weight pricing, certificates, reservations.

`SQL: products, metals, stones, certificates, customers, orders, order_items, gold_prices`
`Mongo: reviews, product_views`

4. Beauty products e-commerce

Cosmetics by skin type and brand, ingredient filter, bundles, loyalty points.

`SQL: brands, products, ingredients, product_ingredients (M:N), customers, orders, loyalty_points`
`Mongo: reviews (with photos), skin_quiz_answers`

5. General e-commerce marketplace marketplace

Many sellers, product approval, cart, orders split per seller, seller sales dashboard.

`SQL: sellers, categories, products, customers, orders, order_items, payouts`
`Mongo: search_logs, click_events`

6. Shoe store retail

Models with size/colour variants and stock per size, size guide, returns.

`SQL: brands, models, variants (size, colour, stock), customers, orders, order_items, returns`
`Mongo: reviews (fit: small/true/large), views`

7. Clothing brand fashion

Collections, variants, wishlist, discount codes, best sellers by season and city.

`SQL: collections, products, variants, customers, wishlists, coupons, orders, order_items`
`Mongo: style_quiz, events`

8. RepetitorHub – tutoring marketplace ed-tech · unique

Find tutors for IELTS, SAT, maths, Russian/Korean; book time slots, rate lessons, tutor earnings report.

`SQL: tutors, subjects, tutor_subjects (M:N), students, time_slots, bookings, payments`
`Mongo: lesson_reviews, search_events`

9. DoriTez – pharmacy stock & delivery health · unique

Search a medicine across pharmacies in your city, see price & stock, reserve or order delivery, prescription flag.

`SQL: pharmacies, medicines, pharmacy_stock (price, qty), customers, orders, order_items, couriers`
`Mongo: search_logs (what is missing where), courier_gps`

10. UstaGo – car repair & rental booking auto · unique

Book a service (oil change, diagnostics) at a garage, or rent a car by the day; garage calendar and invoices.

`SQL: garages, services, mechanics, cars, customers, bookings, rentals, invoices`
`Mongo: car_service_history (flexible), reviews`

11. Hunarmand – local crafts marketplace tourism · unique

Artisans sell suzani, Rishtan ceramics, Bukhara knives, atlas silk; workshop tours booking; export to tourists.

`SQL: artisans, regions, crafts, products, customers, orders, order_items, workshop_tours, tour_bookings`
`Mongo: stories (rich text + photos), visitor_events`

Bonus: To'yxona – wedding & event venue booking events · unique

Compare venues by capacity/price/date, book a date with deposit, add services (music, photo, plov).

`SQL: venues, halls, dates_available, customers, bookings, extra_services, booking_services`
`Mongo: reviews, inquiry_events`

2.1 Turning SavdoShop into your project

The reference store already contains all the patterns you need. Rename and extend – do not start from zero:

SavdoShop table	Restaurant	Bank / wallet	Tutoring (RepetitorHub)	Pharmacy (DoriTez)
`categories`	menu_categories	spending_categories	subjects	medicine_groups
`products`	dishes	accounts	tutors	medicines + pharmacy_stock
`customers`	customers	clients	students	customers
`orders`	orders (table no., status)	transfers	bookings	orders / reservations
`order_items`	order_items	transactions (debit/credit)	booking_slots	order_items
`payments`	payments	card_payments	payments	payments
Mongo `reviews`	dish reviews	support_tickets	lesson_reviews	pharmacy reviews
Mongo `events`	menu_views	login_events	search_events	search_logs

Minimum size of YOUR project (individual)

≥ 6 SQL tables with at least one many-to-many relation · ≥ 30 rows of realistic sample data (Uzbek names, cities, so'm prices) · ≥ 2 MongoDB collections, one with ≥ 10 000 generated documents · ≥ 8 API routes · ≥ 4 web pages · 1 transaction (order, booking, transfer…). **Pairs:** + login with hashed passwords (`bcrypt`) + an admin page to add/edit items.

1 Week 1 – Design & build the database

Goal: a complete PostgreSQL + MongoDB database for your project, filled with data, and 30+ practised queries.

Day 1 (lab)	Day 2	Day 3 (lab)	Day 4	Day 5
Check tools, run SavdoShop scripts, choose idea	Entity list + ER diagram, teacher approves idea	Write your schema SQL, sample data	20 SQL queries, 500k-row index test	MongoDB collections, 12 queries, submit

Step 1 · Check that every tool works (15 min)

Open **Command Prompt** (`Win` + `R`, type `cmd`, `Enter`) and type each command. You should see a version number, not an error.

```
node -v          # v20.x or v22.x
npm -v           # 10.x
git --version    # git version 2.x
psql --version   # psql (PostgreSQL) 16 or 17   (may say "not recognized" - that is OK, we use pgAdmin)
```

Then open **pgAdmin 4** and **MongoDB Compass** from the Start menu. In pgAdmin, click **Servers** → **PostgreSQL 17** and enter the lab password (ask your teacher, often `postgres`). In Compass, click **Connect** with the default address `mongodb://localhost:27017`.

> **If something is missing**
> – go to Chapter 7, rows 1–5. Do not continue until pgAdmin and Compass both connect.

Step 2 · Create the database in pgAdmin

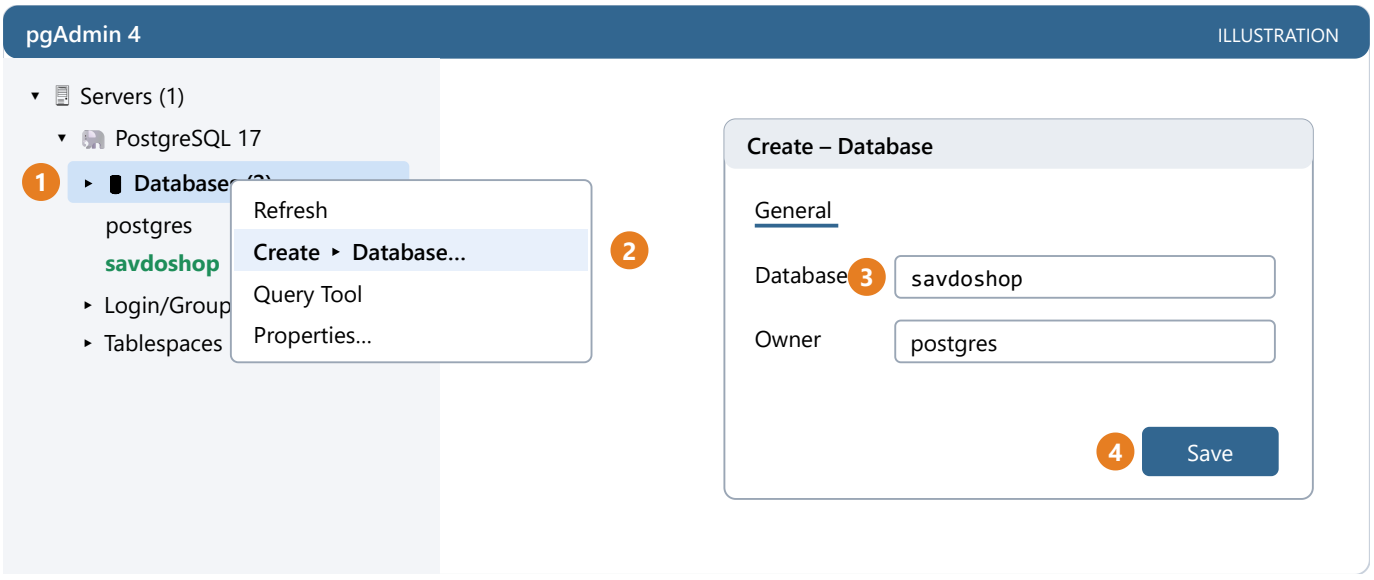


Figure 3.1 – **ILLUSTRATION** pgAdmin: (1) right-click **Databases** (2) **Create** → **Database...** (3) type the name (4) **Save**.

1. In the left tree, right-click **Databases** → **Create** → **Database...**
2. Database name: `savdoshop` (later create a second one for your own project, e.g. `restaurant_db`). Click **Save**.
3. Right-click the new database → **Query Tool**. A SQL editor opens. Everything you run here goes into *this* database – check the name in the tab title.

4. Click the folder icon (**Open File**), choose `BigData_Starter_Code/sql/01_schema.sql` and press `F5` (Execute script). Bottom panel: *"Query returned successfully"*.
5. Repeat with `02_seed.sql`. Then right-click **Schemas → public → Tables → Refresh** : you see 6 tables.
6. Right-click `products` → **View/Edit Data → All Rows** : 12 products appear. 🎉 Your first database works.

Step 3 · Design: entities, relations, ER diagram

Before you write SQL for *your* project, draw it. Use **draw.io** (free, app.diagrams.net) or pgAdmin's **ERD Tool** (right-click the database → **ERD For Database** – it draws the diagram from your tables automatically).

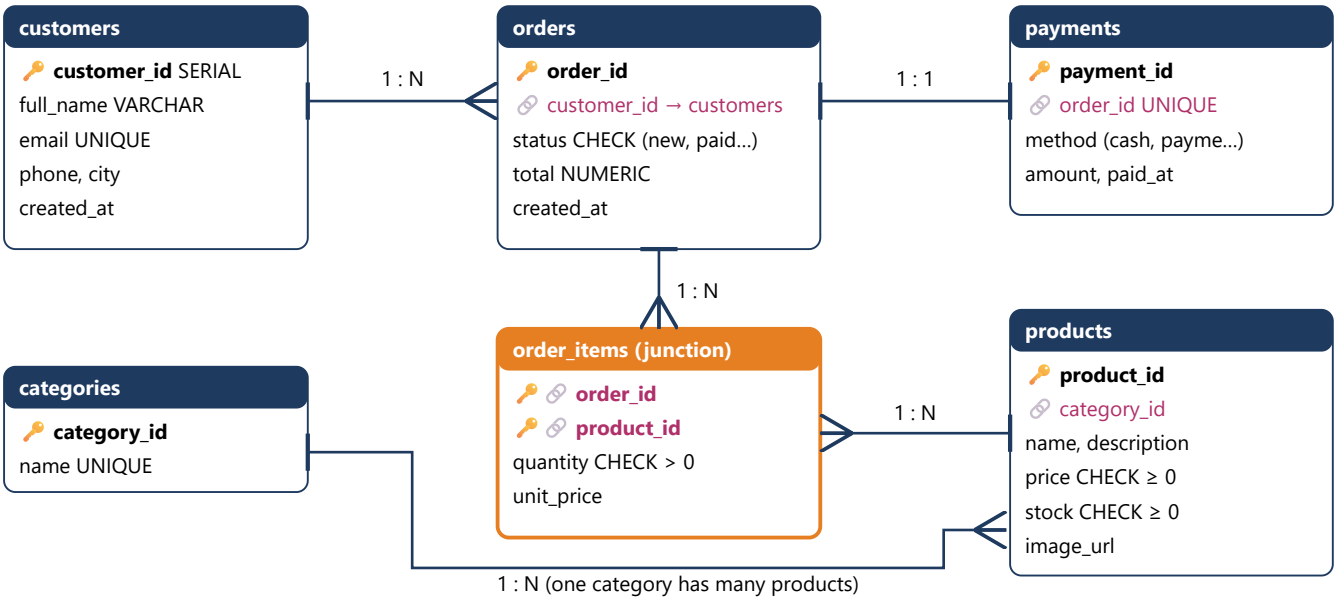


Figure 3.2 – ER diagram of SavdoShop (crow's-foot notation). orders ↔ products is many-to-many, solved with the junction table `order_items`.

Design checklist (normalisation in plain words)

- Every table has a primary key (`SERIAL` id).
- One fact is stored in one place (category *name* lives only in `categories`).
- No lists inside a column – use a new table (M:N → junction table).
- Use `CHECK` , `NOT NULL` , `UNIQUE` so bad data is impossible.
- Copy the price into `order_items` – prices change, old orders must not.

Choosing data types

- Money → `NUMERIC(12,2)` (never `FLOAT` : 0.1+0.2 ≠ 0.3)
- Text → `VARCHAR(n)` for short, `TEXT` for long
- Dates → `TIMESTAMPTZ DEFAULT now()`
- Yes/no → `BOOLEAN` ; fixed lists → `CHECK (x IN (…))`
- Phone numbers → `VARCHAR` (not a number: "+998…")

Step 4 · The schema script explained

This is the exact file you ran in Step 2. Read the comments – your own schema will follow the same pattern.

01_schema.sql SQL

```sql
-- ============================================================
-- SavdoShop reference project - PostgreSQL schema
-- Run in pgAdmin: right-click database "savdoshop" > Query Tool,
-- open this file, press F5 (Execute script).
```

```sql
-- Safe to re-run: it drops the old tables first.
-- ============================================================

DROP TABLE IF EXISTS payments, order_items, orders, products, categories, customers CASCADE;

CREATE TABLE customers (
    customer_id  SERIAL PRIMARY KEY,
    full_name    VARCHAR(100) NOT NULL,
    email        VARCHAR(120) NOT NULL UNIQUE,
    phone        VARCHAR(20),
    city         VARCHAR(50)  NOT NULL DEFAULT 'Tashkent',
    created_at   TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE TABLE categories (
    category_id  SERIAL PRIMARY KEY,
    name         VARCHAR(60) NOT NULL UNIQUE
);

CREATE TABLE products (
    product_id   SERIAL PRIMARY KEY,
    category_id  INT NOT NULL REFERENCES categories(category_id),
    name         VARCHAR(120) NOT NULL,
    description  TEXT,
    price        NUMERIC(12,2) NOT NULL CHECK (price >= 0),   -- in UZS (so'm)
    stock        INT NOT NULL DEFAULT 0 CHECK (stock >= 0),
    image_url    TEXT,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE orders (
    order_id     SERIAL PRIMARY KEY,
    customer_id  INT NOT NULL REFERENCES customers(customer_id),
    status       VARCHAR(20) NOT NULL DEFAULT 'new'
                 CHECK (status IN ('new', 'paid', 'shipped', 'delivered', 'cancelled')),
    total        NUMERIC(12,2) NOT NULL DEFAULT 0,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Junction table: one order has many products, one product is in many orders (M:N)
CREATE TABLE order_items (
    order_id     INT NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE,
    product_id   INT NOT NULL REFERENCES products(product_id),
    quantity     INT NOT NULL CHECK (quantity > 0),
    unit_price   NUMERIC(12,2) NOT NULL,
    PRIMARY KEY (order_id, product_id)
);

CREATE TABLE payments (
    payment_id   SERIAL PRIMARY KEY,
    order_id     INT NOT NULL UNIQUE REFERENCES orders(order_id) ON DELETE CASCADE,
    method       VARCHAR(20) NOT NULL CHECK (method IN ('cash', 'card', 'payme', 'click')),
    amount       NUMERIC(12,2) NOT NULL,
    paid_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Indexes on foreign keys and columns we search/filter on
CREATE INDEX idx_products_category ON products(category_id);
CREATE INDEX idx_orders_customer   ON orders(customer_id);
CREATE INDEX idx_orders_created    ON orders(created_at);
```

Step 5 · Sample data

A database without data cannot be tested. Insert realistic rows – Uzbek names, real cities, prices in so'm. Excerpt:

02_seed.sql SQL

```sql
INSERT INTO categories (name) VALUES
  ('Clothing'), ('Shoes'), ('Beauty'), ('Jewelry'), ('Electronics');

INSERT INTO customers (full_name, email, phone, city) VALUES
  ('Aziza Karimova',    'aziza@example.uz',   '+998901112233', 'Tashkent'),
  ('Bekzod Tursunov',   'bekzod@example.uz',  '+998935556677', 'Samarkand'),
  ('Dilnoza Rahimova',  'dilnoza@example.uz', '+998977778899', 'Bukhara'),
  ('Jasur Aliyev',      'jasur@example.uz',   '+998901234567', 'Tashkent'),
  ('Malika Yusupova',   'malika@example.uz',  '+998991112244', 'Namangan'),
  ('Otabek Ergashev',   'otabek@example.uz',  '+998903334455', 'Andijan'),
  ('Sevara Nazarova',   'sevara@example.uz',  '+998946667788', 'Fergana'),
  ('Timur Saidov',      'timur@example.uz',   '+998909998877', 'Tashkent'),
  ('Zarina Islomova',   'zarina@example.uz',  '+998931231231', 'Khiva'),
```

Fast way to make 30+ rows:
use a spreadsheet: type rows in Excel/Google Sheets, save as CSV, then in pgAdmin right-click the table → Import/Export Data… → Import, Format *csv*, Header *on*.

Step 6 · SQL practice – CRUD, JOINs, aggregation

Open `03_practice_queries.sql` in the Query Tool. **Select one query with the mouse and press** `F5` – only the selected text runs. Take a screenshot of each result for your report.

03_practice_queries.sql SQL

```sql
-- ---------- C R U D ----------
-- Q1 CREATE: add a new customer and return the generated id
INSERT INTO customers (full_name, email, city)
VALUES ('Student Test', 'student@example.uz', 'Tashkent')
RETURNING customer_id;

-- Q2 READ: products cheaper than 400 000 so'm, cheapest first
SELECT product_id, name, price
FROM products
WHERE price < 400000
ORDER BY price ASC;

-- Q3 UPDATE: 10% discount on all Shoes
UPDATE products
SET price = price * 0.9
WHERE category_id = (SELECT category_id FROM categories WHERE name = 'Shoes');

-- Q4 DELETE: remove the test customer
DELETE FROM customers WHERE email = 'student@example.uz';

-- ---------- J O I N S ----------
-- Q5 INNER JOIN: every product with its category name
SELECT p.name AS product, c.name AS category, p.price
FROM products p
JOIN categories c ON c.category_id = p.category_id
ORDER BY c.name, p.name;
```

03_practice_queries.sql (continued) SQL

```sql
-- Q6 Order details: who bought what (4 tables)
SELECT o.order_id, cu.full_name, p.name AS product, oi.quantity,
       oi.unit_price * oi.quantity AS line_total
FROM orders o
JOIN customers   cu ON cu.customer_id = o.customer_id
JOIN order_items oi ON oi.order_id    = o.order_id
```

```sql
JOIN products    p  ON p.product_id   = oi.product_id
ORDER BY o.order_id;

-- Q7 LEFT JOIN: customers who have NEVER ordered
SELECT cu.full_name, cu.city
FROM customers cu
LEFT JOIN orders o ON o.customer_id = cu.customer_id
WHERE o.order_id IS NULL;

-- ---------- A G G R E G A T I O N ----------
-- Q8 Number of products and average price per category
SELECT c.name, COUNT(p.product_id) AS products, ROUND(AVG(p.price)) AS avg_price
FROM categories c
LEFT JOIN products p ON p.category_id = c.category_id
GROUP BY c.name
ORDER BY products DESC;

-- Q9 Revenue per city (only cities with revenue > 500 000)
SELECT cu.city, SUM(o.total) AS revenue, COUNT(*) AS orders
FROM orders o
JOIN customers cu ON cu.customer_id = o.customer_id
WHERE o.status <> 'cancelled'
GROUP BY cu.city
HAVING SUM(o.total) > 500000
ORDER BY revenue DESC;

-- Q10 Best-selling products (by quantity)
SELECT p.name, SUM(oi.quantity) AS sold
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
GROUP BY p.name
ORDER BY sold DESC
LIMIT 5;
```

Understand the JOIN types

– `JOIN` (inner) returns only matching rows; `LEFT JOIN` keeps every row of the left table and fills missing right side with `NULL`. That is why Q7 ("customers who never ordered") uses `LEFT JOIN … WHERE o.order_id IS NULL`.

Views, window functions and a transaction

03_practice_queries.sql (end) SQL

```sql
-- ---------- V I E W S  &  W I N D O W  F U N C T I O N S ----------
-- Q11 A view the website can reuse
CREATE OR REPLACE VIEW v_product_catalog AS
SELECT p.product_id, p.name, p.price, p.stock, p.image_url, c.name AS category
FROM products p
JOIN categories c ON c.category_id = p.category_id;

SELECT * FROM v_product_catalog WHERE category = 'Jewelry';

-- Q12 Rank products by price inside each category
SELECT category, name, price,
       RANK() OVER (PARTITION BY category ORDER BY price DESC) AS price_rank
FROM v_product_catalog;

-- ---------- T R A N S A C T I O N ----------
-- Q13 Place an order safely: all steps succeed or none do
BEGIN;
  INSERT INTO orders (customer_id) VALUES (3) RETURNING order_id;
  -- currval() = the order_id that was just generated in this session
  INSERT INTO order_items (order_id, product_id, quantity, unit_price)
  SELECT currval('orders_order_id_seq'), product_id, 1, price
```

```
  FROM products WHERE product_id = 4;
  UPDATE products SET stock = stock - 1 WHERE product_id = 4;
  UPDATE orders SET total = (SELECT SUM(quantity * unit_price) FROM order_items
                             WHERE order_id = currval('orders_order_id_seq'))
  WHERE order_id = currval('orders_order_id_seq');
COMMIT;   -- or ROLLBACK; to cancel everything
```

A **transaction** (`BEGIN … COMMIT`) is the most important idea for business apps: the order, its items and the stock change are saved **together or not at all**. If the computer crashes in the middle, PostgreSQL rolls everything back. Your Week 2 API does exactly this in JavaScript.

Step 7 · Big data in PostgreSQL: 500 000 rows and indexes

Now the "Big Data" part: generate half a million sales with one statement, then measure how an index changes speed.

04_big_data.sql — SQL

```sql
CREATE TABLE sales_log (
    id          BIGSERIAL PRIMARY KEY,
    product_id  INT NOT NULL,
    city        VARCHAR(50) NOT NULL,
    amount      NUMERIC(12,2) NOT NULL,
    sold_at     TIMESTAMPTZ NOT NULL
);

-- Step 1: generate 500 000 random sales in one statement
INSERT INTO sales_log (product_id, city, amount, sold_at)
SELECT 1 + floor(random() * 12)::int,
       (ARRAY['Tashkent','Samarkand','Bukhara','Namangan','Andijan',
              'Fergana','Khiva','Nukus'])[1 + floor(random() * 8)::int],
       round((50000 + random() * 2000000)::numeric, -3),
       now() - random() * interval '365 days'
FROM generate_series(1, 500000);

ANALYZE sales_log;           -- refresh planner statistics
SELECT COUNT(*) FROM sales_log;

-- Step 2: measure a query WITHOUT an index (look for "Seq Scan" + Execution Time)
EXPLAIN ANALYZE
SELECT COUNT(*), SUM(amount)
FROM sales_log
WHERE city = 'Khiva' AND sold_at > now() - interval '30 days';

-- Step 3: add a composite index and measure again (Look for "Index" / "Bitmap")
CREATE INDEX idx_sales_city_date ON sales_log (city, sold_at);

EXPLAIN ANALYZE
SELECT COUNT(*), SUM(amount)
FROM sales_log
WHERE city = 'Khiva' AND sold_at > now() - interval '30 days';
-- Write both execution times in your report and explain the difference.
```

Before the index

```
Aggregate  (actual time=46.0..46.0 rows=1)
  ->  Seq Scan on sales_log
        Filter: city = 'Khiva' AND sold_at > ...
        Rows Removed by Filter: 494897
Execution Time: 46.107 ms
```

After CREATE INDEX

```
Aggregate  (actual time=5.0..5.0 rows=1)
  ->  Bitmap Heap Scan on sales_log
        ->  Bitmap Index Scan on idx_sales_city_date
              Index Cond: city = 'Khiva' AND ...
Execution Time: 5.054 ms
```

Figure 3.3 – **REAL OUTPUT** from our test run: the index makes the query ~9× faster (your numbers will differ slightly). Without it PostgreSQL reads all 500 000 rows ("Seq Scan").

04_big_data.sql (analytics) SQL

```sql
-- Step 4: monthly revenue trend (time-series analytics)
SELECT date_trunc('month', sold_at)::date AS month,
       COUNT(*)            AS sales,
       ROUND(SUM(amount))  AS revenue
FROM sales_log
GROUP BY 1
ORDER BY 1;

-- Step 5: materialized view = pre-computed summary for fast dashboards
DROP MATERIALIZED VIEW IF EXISTS mv_city_daily;
CREATE MATERIALIZED VIEW mv_city_daily AS
SELECT city, sold_at::date AS day, COUNT(*) AS sales, SUM(amount) AS revenue
FROM sales_log
GROUP BY city, sold_at::date;

SELECT * FROM mv_city_daily WHERE city = 'Tashkent' ORDER BY day DESC LIMIT 7;
REFRESH MATERIALIZED VIEW mv_city_daily;   -- run again after new data arrives

-- Step 6: top 3 products in every city (window function over 500k rows)
SELECT * FROM (
    SELECT city, product_id, SUM(amount) AS revenue,
           ROW_NUMBER() OVER (PARTITION BY city ORDER BY SUM(amount) DESC) AS rn
    FROM sales_log
    GROUP BY city, product_id
) t
WHERE rn <= 3
ORDER BY city, rn;
```

Step 8 · MongoDB: documents, collections, aggregation

MongoDB stores **documents** (JSON objects) in **collections**. No fixed schema: one review may have photos, another not. That makes it perfect for reviews, logs and events.

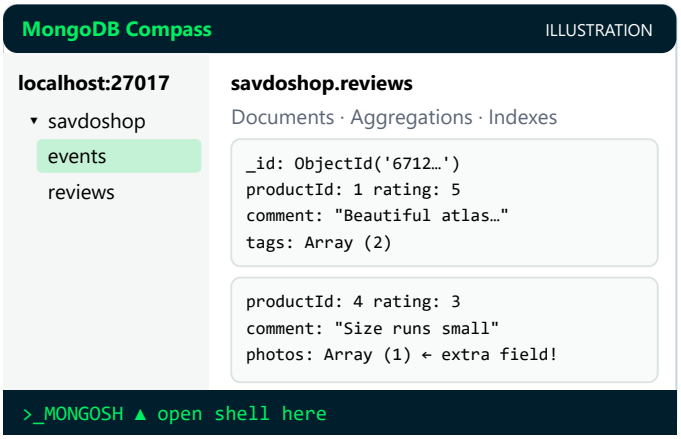


Figure 3.4 – **ILLUSTRATION** Compass: database → collections → documents; the **_MONGOSH** bar at the bottom opens a shell.

SQL word	MongoDB word
database	database
table	collection
row	document (JSON)
column	field
JOIN	`$lookup` (or embed)
GROUP BY	`$group`
WHERE	`find({...})` / `$match`

Three ways to run the MongoDB scripts (choose one)

1. **VS Code (easiest):** install the extension *MongoDB for VS Code*, connect to `mongodb://localhost:27017`, open `01_setup.mongodb.js` and click ▶ Run (top-right). Files ending in `.mongodb.js` are "playgrounds".
2. **Compass:** click _MONGOSH at the bottom, open the .js file in Notepad, copy everything, paste into the shell, press `Enter`.
3. **Terminal:** `mongosh "mongodb://localhost:27017" --file 01_setup.mongodb.js` (if `mongosh` is installed).

01_setup.mongodb.js JAVASCRIPT

```javascript
use("savdoshop");

db.reviews.drop();
db.events.drop();

// 1) Reviews: flexible documents (tags array, optional photos)
db.reviews.insertMany([
  { productId: 1,  customerId: 1, rating: 5, comment: "Beautiful atlas, fast delivery", tags: ["quality",
"delivery"], createdAt: new Date() },
  { productId: 1,  customerId: 3, rating: 4, comment: "Nice colours, a bit long",      tags: ["size"],
createdAt: new Date() },
  { productId: 4,  customerId: 2, rating: 5, comment: "Very comfortable",               tags: ["comfort"],
createdAt: new Date() },
  { productId: 4,  customerId: 6, rating: 3, comment: "Size runs small",                tags: ["size"], photos:
["s1.jpg"], createdAt: new Date() },
  { productId: 7,  customerId: 2, rating: 4, comment: "Smells great",                   tags: ["quality"],
createdAt: new Date() },
  { productId: 10, customerId: 4, rating: 5, comment: "Perfect gift",                   tags: ["gift", "quality"],
createdAt: new Date() },
  { productId: 11, customerId: 8, rating: 2, comment: "Battery weaker than promised",   tags: ["battery"],
createdAt: new Date() }
]);

// 2) Events: clickstream - the "big" data (50 000 documents)
const types   = ["view", "view", "view", "view", "add_to_cart", "add_to_cart", "purchase"];
const cities  = ["Tashkent", "Samarkand", "Bukhara", "Namangan", "Andijan", "Fergana", "Khiva", "Nukus"];
const devices = ["android", "ios", "desktop"];
const pick = (arr) => arr[Math.floor(Math.random() * arr.length)];

let batch = [];
for (let i = 0; i < 50000; i++) {
  batch.push({
    type: pick(types),
    productId: 1 + Math.floor(Math.random() * 12),
    customerId: 1 + Math.floor(Math.random() * 10),
    city: pick(cities),
    device: pick(devices),
    ts: new Date(Date.now() - Math.floor(Math.random() * 90 * 24 * 3600 * 1000))
  });
  if (batch.length === 5000) { db.events.insertMany(batch); batch = []; }
}
if (batch.length) db.events.insertMany(batch);

// 3) Indexes
db.reviews.createIndex({ productId: 1, createdAt: -1 });
db.events.createIndex({ type: 1, ts: -1 });
db.events.createIndex({ productId: 1 });

print("reviews:", db.reviews.countDocuments(), " events:", db.events.countDocuments());
```

Expected output: `reviews: 7 events: 50000`. In Compass click **Refresh** – two collections appear.

MongoDB query practice

02_queries.mongodb.js JAVASCRIPT

```javascript
// ---------- C R U D ----------
// M1 CREATE
db.reviews.insertOne({ productId: 2, customerId: 5, rating: 5, comment: "Great shirt", tags: ["quality"],
createdAt: new Date() });

// M2 READ: all 5-star reviews, newest first, only some fields
db.reviews.find({ rating: 5 }, { _id: 0, productId: 1, comment: 1 }).sort({ createdAt: -1 });
```

```
// M3 READ with operators: rating 3 or lower OR tagged "size"
db.reviews.find({ $or: [{ rating: { $lte: 3 } }, { tags: "size" }] });

// M4 UPDATE: add a tag to one review, increase "helpful" counter
db.reviews.updateOne({ productId: 2, customerId: 5 }, { $push: { tags: "recommended" }, $inc: { helpful: 1 } });

// M5 DELETE
db.reviews.deleteOne({ productId: 2, customerId: 5 });
```

02_queries.mongodb.js (aggregation + indexes) JAVASCRIPT

```
db.events.aggregate([
  { $group: { _id: "$type", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
]);

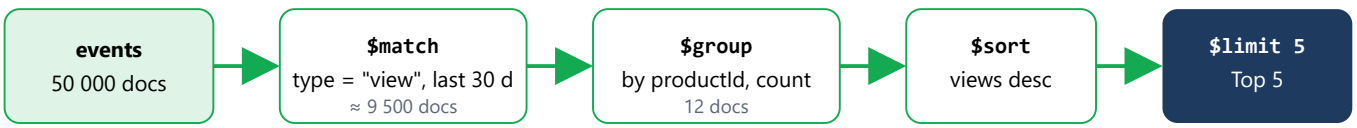
// M8 Top 5 most viewed products in the last 30 days
db.events.aggregate([
  { $match: { type: "view", ts: { $gte: new Date(Date.now() - 30 * 24 * 3600 * 1000) } } },
  { $group: { _id: "$productId", views: { $sum: 1 } } },
  { $sort: { views: -1 } },
  { $limit: 5 }
]);

// M9 Purchases per city and device (two-field group key)
db.events.aggregate([
  { $match: { type: "purchase" } },
  { $group: { _id: { city: "$city", device: "$device" }, n: { $sum: 1 } } },
  { $sort: { "_id.city": 1, n: -1 } }
]);

// M10 Daily activity (time series) for the last 14 days
db.events.aggregate([
  { $match: { ts: { $gte: new Date(Date.now() - 14 * 24 * 3600 * 1000) } } },
  { $group: { _id: { $dateToString: { format: "%Y-%m-%d", date: "$ts" } }, events: { $sum: 1 } } },
  { $sort: { _id: 1 } }
]);

// M11 Most common review tags ($unwind turns an array into separate rows)
db.reviews.aggregate([
  { $unwind: "$tags" },
  { $group: { _id: "$tags", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
]);

// ---------- I N D E X E S ----------
// M12 Compare: look at "totalDocsExamined" and "executionTimeMillis"
db.events.find({ city: "Khiva", type: "purchase" }).explain("executionStats").executionStats;
db.events.createIndex({ city: 1, type: 1 });
db.events.find({ city: "Khiva", type: "purchase" }).explain("executionStats").executionStats;
db.events.getIndexes();
```



Each stage receives the documents from the previous stage – like an assembly line.

Figure 3.5 – How the aggregation pipeline M8 (“Top 5 viewed products”) works.

Your turn – Week 1 tasks for YOUR project

1. Create a new database (e.g. `pharmacy_db`) and write `01_schema.sql` for your idea (≥ 6 tables, ≥ 1 M:N, CHECK constraints, indexes on foreign keys).
2. Write `02_seed.sql` with ≥ 30 realistic rows in total (≥ 5 in every table).
3. Write **20 SQL queries** in `03_queries.sql`: 4 CRUD, 5 JOIN (one LEFT JOIN), 4 GROUP BY (one HAVING), 1 view, 2 window functions, 1 subquery, 1 transaction, 2 of your choice. Above each query write a comment: *which business question does it answer?*
4. Adapt `04_big_data.sql` to your domain (e.g. 500k pharmacy searches, 500k bank transactions) and record the time before/after the index.
5. Create ≥ 2 MongoDB collections (one with ≥ 10 000 generated documents) and write **12 queries**: 5 CRUD, 5 aggregations, 2 index/explain.

📦 **Deliverable 1 (end of Week 1)** – a folder `week1/` pushed to GitHub (or zipped): `01_schema.sql`, `02_seed.sql`, `03_queries.sql`, `04_big_data.sql`, `mongo_setup.js`, `mongo_queries.js`, `er_diagram.png`, and `week1_results.pdf` with a screenshot of each query result.

Week 1 grading checklist (30 points)

Criterion	Pts	What the teacher checks
ER diagram	**4**	All tables, PK/FK, cardinalities (1:N, M:N) correct
Schema quality	**7**	≥ 6 tables, correct types (NUMERIC for money), NOT NULL/UNIQUE/CHECK, FKs, indexes; script runs without error on a fresh database
Sample data	**4**	≥ 30 realistic rows, no FK errors
20 SQL queries	**7**	All categories present, correct results, comments explain the business question
Index experiment	**3**	500k rows, EXPLAIN ANALYZE before/after, short explanation
MongoDB	**5**	2 collections, ≥ 10k docs, 12 queries incl. 5 aggregations

2 Week 2 – Build the website & API

Goal: a Node.js/Express website on `http://localhost:3000` that reads and writes both databases.

Day 6 (lab)	Day 7	Day 8 (lab)	Day 9	Day 10
Run SavdoShop server, understand files	Copy & rename for your project, .env, list routes	Write your API routes, test with Thunder Client	HTML pages + app.js fetch, transaction route	Dashboard, polish, GitHub, submit

Step 1 · Project structure

```
server/
├─ package.json        ← project name, dependencies, "npm start"
├─ .env.example        ← template for your secrets
├─ .env                ← YOUR secrets (never upload!)
├─ .gitignore          ← tells Git to skip node_modules and .env
├─ src/
│  ├─ db.js            ← connects to PostgreSQL + MongoDB
│  └─ server.js        ← all API routes (Express)
├─ public/             ← the website (served as static files)
│  ├─ index.html       ← product list + search
│  ├─ product.html     ← detail + reviews
│  ├─ cart.html        ← cart + checkout
│  ├─ dashboard.html   ← analytics
│  ├─ app.js           ← front-end JavaScript
│  └─ style.css
└─ node_modules/       ← created by "npm install"
```

Request journey (what happens when you open a product page):

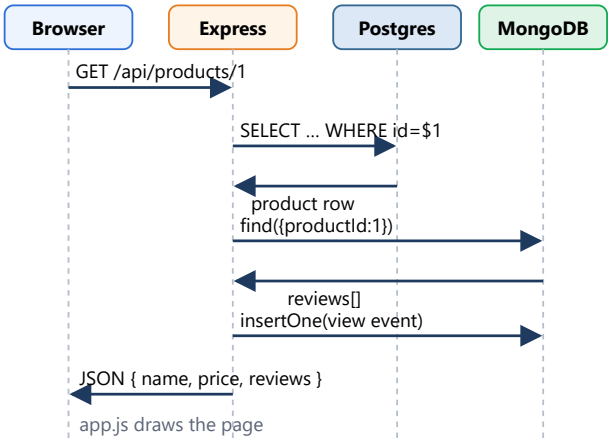


Figure 4.1 – One request uses both databases.

Step 2 · Install and start the reference server

1. Copy `BigData_Starter_Code` to `D:\Projects\` (or your Documents). Open the `server` folder in VS Code: **File → Open Folder...**
2. Open the terminal in VS Code: **Terminal → New Terminal** (`Ctrl`+`` ` ``) and install the libraries:

```
npm install
```

3. Create your secrets file: copy `.env.example` and name the copy `.env`. Put the lab PostgreSQL password in it:

.env (copy of .env.example) — INI

```
# --- Local lab computer ---
DATABASE_URL=postgres://postgres:YOUR_PASSWORD@localhost:5432/savdoshop
PGSSL=false
MONGODB_URI=mongodb://127.0.0.1:27017
MONGODB_DB=savdoshop
PORT=3000
```

4. Start the server:

```
npm run dev          # restarts automatically when you save a file  (or: npm start)
# SavdoShop running on http://localhost:3000
```

5 Open `http://localhost:3000/api/health` in Chrome. You should see `{"ok":true,"postgresTime":"…","mongoReviews":7}`. Then open `http://localhost:3000`.

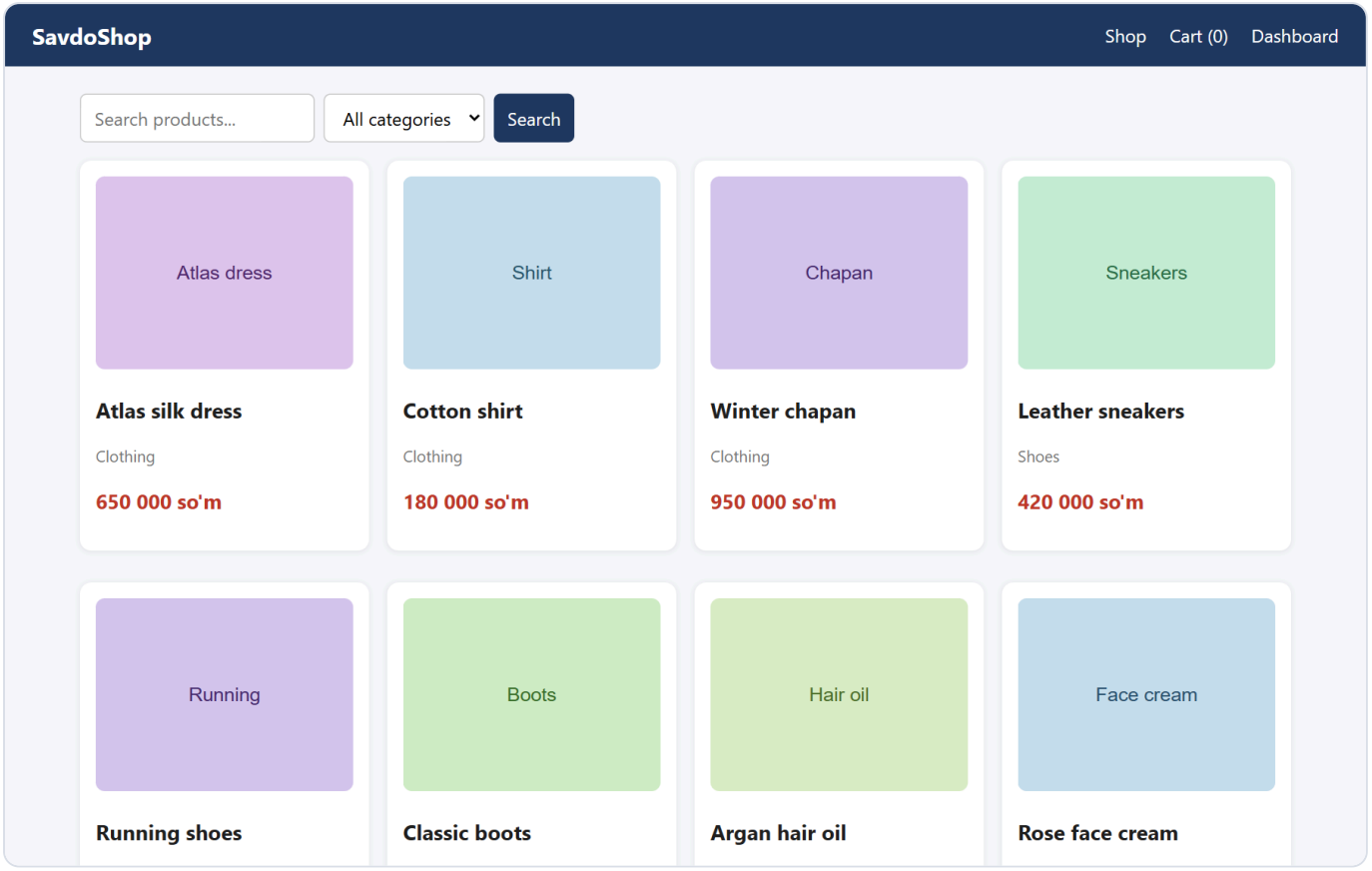


Figure 4.2 – **REAL SCREENSHOT** The SavdoShop home page served by Express. Products come from PostgreSQL via `GET /api/products`.

Step 3 · Connecting to the databases – `db.js`

db.js — JAVASCRIPT

```
// Database connections: PostgreSQL (pg Pool) + MongoDB (official driver)
import 'dotenv/config';
import pg from 'pg';
import { MongoClient } from 'mongodb';

if (!process.env.DATABASE_URL || !process.env.MONGODB_URI) {
  console.error('Missing DATABASE_URL or MONGODB_URI. Did you create the .env file?');
  process.exit(1);
}

// A pool keeps a few connections open and reuses them for every request
export const pool = new pg.Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: process.env.PGSSL === 'true' ? { rejectUnauthorized: false } : false,
  max: 5
});

const mongoClient = new MongoClient(process.env.MONGODB_URI);
let mongoDb = null;

export async function connectMongo() {
  await mongoClient.connect();
  mongoDb = mongoClient.db(process.env.MONGODB_DB || 'savdoshop');
  return mongoDb;
}
```

```
export function mongo() {
  if (!mongoDb) throw new Error('MongoDB is not connected yet');
  return mongoDb;
}
```

> **Pool vs. client:**
> a `Pool` keeps up to 5 connections open and lends one to each request – much faster than connecting every time. For a transaction we borrow **one** client with `pool.connect()` so that BEGIN, the inserts and COMMIT all run on the same connection.

Step 4 · API routes – `server.js`

4a. Setup, health check and the product list (search + filter + paging)

server.js — JAVASCRIPT

```
import express from 'express';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import { pool, connectMongo, mongo } from './db.js';

const app = express();
const __dirname = path.dirname(fileURLToPath(import.meta.url));

app.use(express.json());                                    // read JSON bodies
app.use(express.static(path.join(__dirname, '..', 'public'))); // serve the website

// Small helper so every async route sends errors to the error handler
const wrap = (fn) => (req, res, next) => fn(req, res, next).catch(next);
```

server.js – product list — JAVASCRIPT

```
// ---------- Product list with search, filter, pagination ----------
// GET /api/products?q=silk&category=1&page=1
app.get('/api/products', wrap(async (req, res) => {
  const q = req.query.q ? `%${req.query.q}%` : null;
  const category = req.query.category ? Number(req.query.category) : null;
  const page = Math.max(1, Number(req.query.page) || 1);
  const pageSize = 12;

  // $1, $2 ... are parameters: the driver escapes them (protects against SQL injection)
  const { rows } = await pool.query(
    `SELECT p.product_id, p.name, p.price, p.stock, p.image_url, c.name AS category
       FROM products p
       JOIN categories c ON c.category_id = p.category_id
      WHERE ($1::text IS NULL OR p.name ILIKE $1)
        AND ($2::int  IS NULL OR p.category_id = $2)
      ORDER BY p.product_id
      LIMIT $3 OFFSET $4`,
    [q, category, pageSize, (page - 1) * pageSize]
  );
  res.json(rows);
}));
```

Security: never build SQL with + or template strings!

❌ `pool.query("SELECT * FROM products WHERE name = '" + req.query.q + "'")` → a user typing `' OR 1=1; DROP TABLE products; --` destroys your database (**SQL injection**).
✅ `pool.query('SELECT * FROM products WHERE name = $1', [req.query.q])` → the driver sends the value separately; it can never become SQL.

4b. Product detail: PostgreSQL + MongoDB in one response

server.js – product detail — JAVASCRIPT

```
// ---------- Product detail: PostgreSQL row + MongoDB reviews ----------
app.get('/api/products/:id', wrap(async (req, res) => {
  const id = Number(req.params.id);
  const { rows } = await pool.query(
    `SELECT p.*, c.name AS category FROM products p
       JOIN categories c ON c.category_id = p.category_id
      WHERE p.product_id = $1`, [id]);
  if (rows.length === 0) return res.status(404).json({ error: 'Product not found' });

  const reviews = await mongo().collection('reviews')
    .find({ productId: id }, { projection: { _id: 0 } })
    .sort({ createdAt: -1 }).limit(20).toArray();
  const avg = reviews.length
    ? reviews.reduce((s, r) => s + r.rating, 0) / reviews.length : null;

  // log a "view" event for analytics (do not wait for it)
  mongo().collection('events').insertOne({ type: 'view', productId: id, ts: new Date(), device: 'web' })
    .catch((e) => console.error('event log failed', e.message));

  res.json({ ...rows[0], reviews, avgRating: avg });
}));
```

4c. Placing an order – a transaction in JavaScript

server.js – POST /api/orders — JAVASCRIPT

```
// ---------- Place an order: one SQL transaction ----------
// body: { customer: {fullName, email, phone, city}, items: [{productId, quantity}], paymentMethod }
app.post('/api/orders', wrap(async (req, res) => {
  const { customer, items, paymentMethod = 'cash' } = req.body;
  if (!customer?.email || !customer?.fullName || !Array.isArray(items) || items.length === 0) {
    return res.status(400).json({ error: 'Customer name, email and at least one item are required' });
  }
  if (!['cash', 'card', 'payme', 'click'].includes(paymentMethod)) {
    return res.status(400).json({ error: 'Unknown payment method' });
  }

  const client = await pool.connect();   // all queries below use the SAME connection
  try {
    await client.query('BEGIN');

    const c = await client.query(
      `INSERT INTO customers (full_name, email, phone, city)
       VALUES ($1, $2, $3, COALESCE($4, 'Tashkent'))
       ON CONFLICT (email) DO UPDATE SET full_name = EXCLUDED.full_name
       RETURNING customer_id`,
      [customer.fullName, customer.email, customer.phone || null, customer.city || null]);
    const customerId = c.rows[0].customer_id;

    const o = await client.query(
      'INSERT INTO orders (customer_id) VALUES ($1) RETURNING order_id', [customerId]);
```

```
    const orderId = o.rows[0].order_id;

    let total = 0;
    for (const it of items) {
      const qty = Number(it.quantity);
      if (!Number.isInteger(qty) || qty < 1) throw Object.assign(new Error('Bad quantity'), { status: 400 });
      // decrease stock only if enough is left - atomic check
      const p = await client.query(
        `UPDATE products SET stock = stock - $1
          WHERE product_id = $2 AND stock >= $1
          RETURNING price, name`, [qty, it.productId]);
      if (p.rowCount === 0) {
        throw Object.assign(new Error(`Not enough stock for product ${it.productId}`), { status: 409 });
      }
      const price = Number(p.rows[0].price);
      total += price * qty;
      await client.query(
        'INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES ($1, $2, $3, $4)',
        [orderId, it.productId, qty, price]);
    }

    await client.query('UPDATE orders SET total = $1 WHERE order_id = $2', [total, orderId]);
    await client.query(
      'INSERT INTO payments (order_id, method, amount) VALUES ($1, $2, $3)',
      [orderId, paymentMethod, total]);

    await client.query('COMMIT');

    // analytics events go to MongoDB after the money part is safe
    mongo().collection('events').insertMany(items.map((it) => ({
      type: 'purchase', productId: Number(it.productId), customerId, city: customer.city || 'Tashkent',
      device: 'web', ts: new Date()
    }))).catch((e) => console.error('event log failed', e.message));

    res.status(201).json({ orderId, total });
  } catch (err) {
    await client.query('ROLLBACK');      // undo EVERYTHING from this order
    throw err;
  } finally {
    client.release();                    // give the connection back to the pool
  }
}));
```

Notice `UPDATE … WHERE stock >= $1`: the check and the change happen in **one** statement, so two customers can never buy the last item at the same time. If any step fails, `catch` runs `ROLLBACK` and the customer gets HTTP 409 “Not enough stock”. (We tested this: the stock and order count stay unchanged.)

4d. Analytics routes for the dashboard

server.js – statistics — JAVASCRIPT

```
// ---------- Dashboard statistics ----------
app.get('/api/stats/sales-by-city', wrap(async (req, res) => {
  const { rows } = await pool.query(
    `SELECT cu.city, COUNT(*)::int AS orders, SUM(o.total)::float AS revenue
       FROM orders o JOIN customers cu ON cu.customer_id = o.customer_id
      WHERE o.status <> 'cancelled'
      GROUP BY cu.city ORDER BY revenue DESC`);
  res.json(rows);
}));

app.get('/api/stats/top-viewed', wrap(async (req, res) => {
  const top = await mongo().collection('events').aggregate([
    { $match: { type: 'view' } },
    { $group: { _id: '$productId', views: { $sum: 1 } } },
```

```
    { $sort: { views: -1 } },
    { $limit: 5 }
  ]).toArray();
  // enrich with product names from PostgreSQL
  const ids = top.map((t) => t._id);
  const { rows } = await pool.query(
    'SELECT product_id, name FROM products WHERE product_id = ANY($1::int[])', [ids]);
  const names = Object.fromEntries(rows.map((r) => [r.product_id, r.name]));
  res.json(top.map((t) => ({ productId: t._id, name: names[t._id] || '?', views: t.views })));
}));

app.get('/api/stats/funnel', wrap(async (req, res) => {
  const rows = await mongo().collection('events').aggregate([
    { $group: { _id: '$type', count: { $sum: 1 } } }
  ]).toArray();
  res.json(Object.fromEntries(rows.map((r) => [r._id, r.count])));
}));

// ---------- Errors ----------
```

Step 5 · Test the API before writing pages

Use the browser for GET requests and **Thunder Client** (VS Code extension) for POST requests.

```
[
  {
    "product_id": 1,
    "name": "Atlas silk dress",
    "price": "650000.00",
    "stock": 14,
    "image_url": "/placeholder.svg?text=Atlas+dress",
    "category": "Clothing"
  }
]
```

Figure 4.3 – **REAL** Response of `GET /api/products?q=silk` in the browser.

Thunder Client: New Request → method **POST** → URL `http://localhost:3000/api/orders` → Body → JSON :

```
{
  "customer": { "fullName": "Aziza Karimova",
"email": "aziza@example.uz", "city": "Khiva" },
  "items": [ { "productId": 1, "quantity": 2 }, {
"productId": 7, "quantity": 1 } ],
  "paymentMethod": "click"
}
```

Expected: status **201**, body `{"orderId":4,"total":1395000}`. Send it with `"quantity": 99` → status **409**.

Step 6 · The front-end: HTML + fetch()

Each page is plain HTML with empty containers; `app.js` calls the API and fills them. Example – the home page:

index.html — XML

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>SavdoShop</title>
  <link rel="stylesheet" href="style.css">
</head>
<body data-page="home">
  <header>
    <a href="/" class="logo">SavdoShop</a>
    <nav><a href="/">Shop</a> <a href="/cart.html">Cart (<span id="cart-count">0</span>)</a> <a
href="/dashboard.html">Dashboard</a></nav>
  </header>
  <main>
    <form id="filters">
```

```
      <input id="q" placeholder="Search products...">
      <select id="category"><option value="">All categories</option></select>
      <button>Search</button>
    </form>
    <div id="products" class="grid"></div>
  </main>
  <script src="app.js"></script>
</body>
</html>
```

public/app.js – helpers and home page — JAVASCRIPT

```
const som = (n) => Number(n).toLocaleString('ru-RU') + " so'm";
// escape user text before putting it into HTML (prevents XSS attacks)
const esc = (s) => String(s ?? '').replace(/[&<>"']/g, (c) =>
  ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }[c]));

async function api(url, options = {}) {
  const res = await fetch(url, { headers: { 'Content-Type': 'application/json' }, ...options });
  const data = await res.json();
  if (!res.ok) throw new Error(data.error || res.statusText);
  return data;
}

// ---------- Cart is stored in the browser (localStorage) ----------
const getCart = () => JSON.parse(localStorage.getItem('cart') || '[]');
const saveCart = (cart) => { localStorage.setItem('cart', JSON.stringify(cart)); updateCartCount(); };
function updateCartCount() {
  const el = document.getElementById('cart-count');
  if (el) el.textContent = getCart().reduce((s, i) => s + i.quantity, 0);
}
function addToCart(p) {
  const cart = getCart();
  const line = cart.find((i) => i.productId === p.product_id);
  if (line) line.quantity++;
  else cart.push({ productId: p.product_id, name: p.name, price: Number(p.price), quantity: 1 });
  saveCart(cart);
}

// ---------- Home page ----------
async function loadHome() {
  const cats = await api('/api/categories');
  document.getElementById('category').innerHTML += cats
    .map((c) => `<option value="${c.category_id}">${esc(c.name)}</option>`).join('');

  async function search() {
    const params = new URLSearchParams({
      q: document.getElementById('q').value,
      category: document.getElementById('category').value
    });
    const products = await api('/api/products?' + params);
    document.getElementById('products').innerHTML = products.map((p) => `
      <a class="card product" href="/product.html?id=${p.product_id}">
        <img src="${esc(p.image_url)}" alt="${esc(p.name)}">
        <h3>${esc(p.name)}</h3>
        <p class="muted">${esc(p.category)}</p>
        <p class="price">${som(p.price)}</p>
      </a>`).join('') || '<p>No products found.</p>';
  }
  document.getElementById('filters').addEventListener('submit', (e) => { e.preventDefault(); search(); });
  search();
}

// ---------- Product page ----------
```

Security #2 – XSS:
review text is typed by strangers. If you put it into `innerHTML` directly, someone can post `<script>` and steal your users' data. The `esc()` helper converts `<` to `<` – always use it for user text.

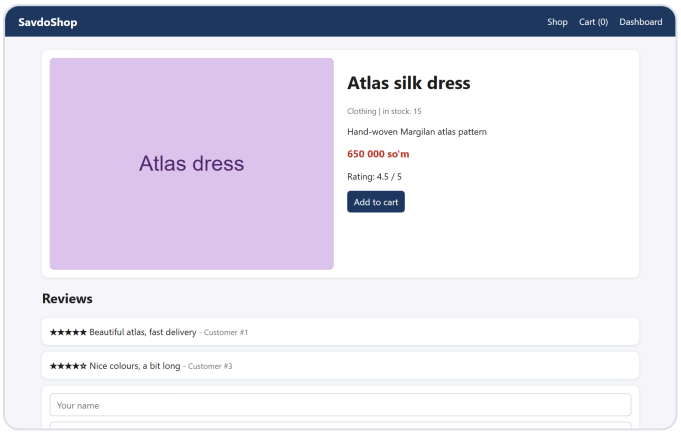


Figure 4.4 – REAL Product page: data from PostgreSQL, reviews and rating from MongoDB.

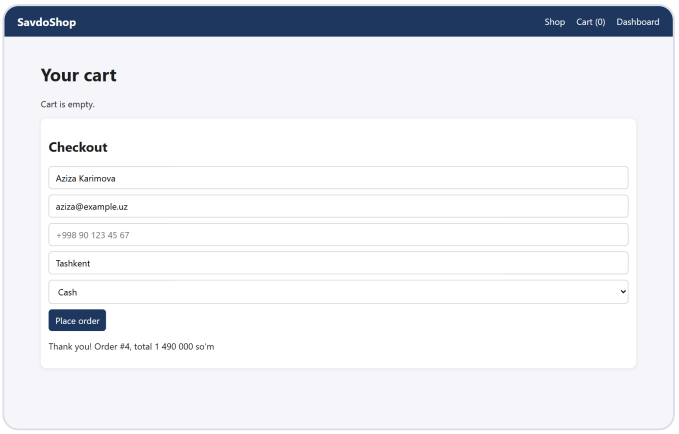


Figure 4.5 – REAL Checkout calls `POST /api/orders`; the transaction returns order #4.

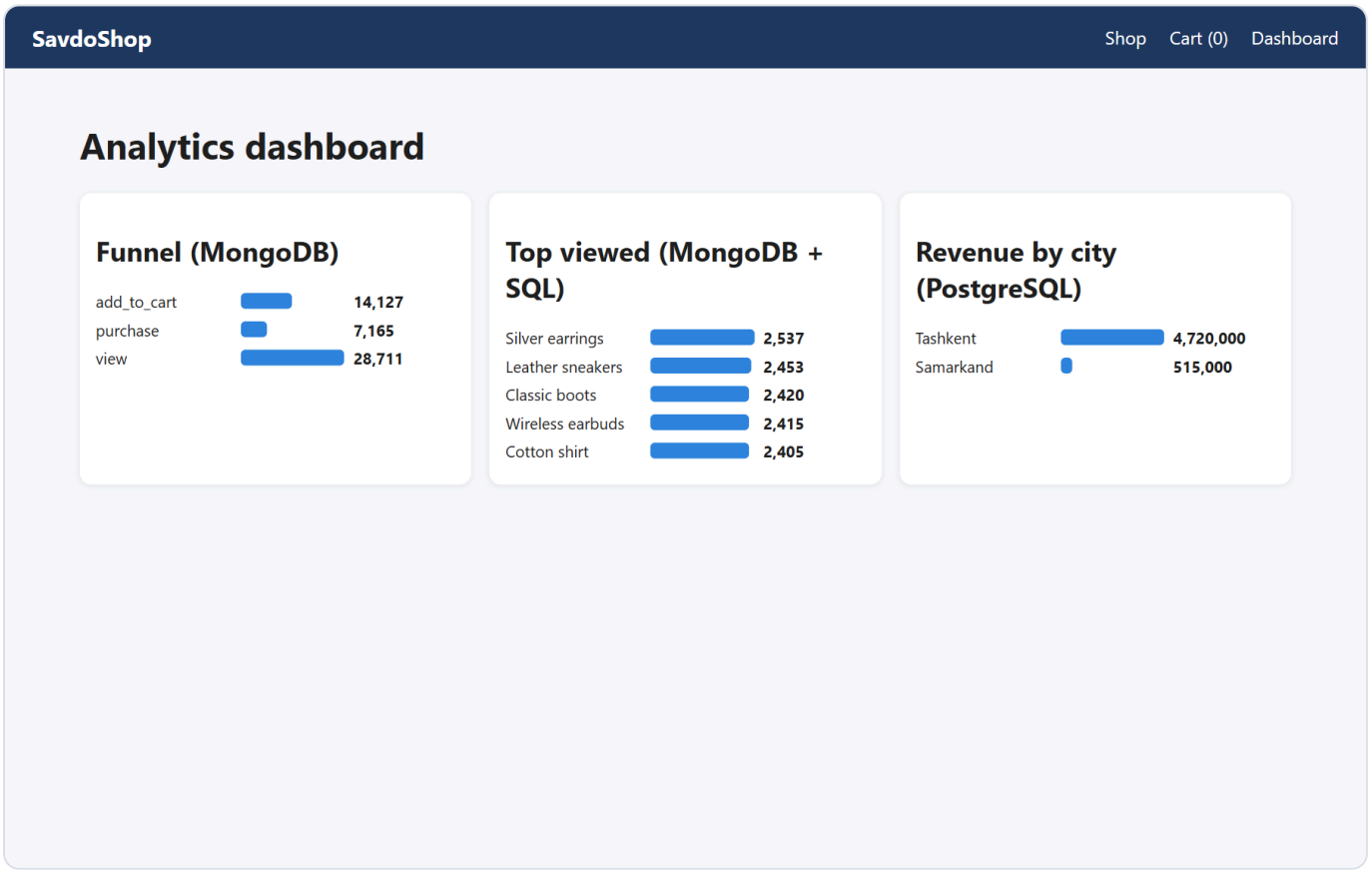


Figure 4.6 – REAL Analytics dashboard: the funnel and "top viewed" come from MongoDB aggregations over 50 000 events; revenue by city from a SQL GROUP BY.

Step 7 · Put your project on GitHub

1. On github.com click **New repository**, name it e.g. `doritez-bigdata`, keep it **Public** (Render needs access), do **not** add a README.

2. In the VS Code terminal, inside your project folder (the one that contains `server/` and `sql/`):

```
git init
git add .
git commit -m "Week 2: API and website"
git branch -M main
git remote add origin https://github.com/YOUR-NAME/doritez-bigdata.git
git push -u origin main
```

3. Open the repository page and check: `.env` and `node_modules` must **not** be there (the `.gitignore` file prevents it).

Your turn – Week 2 tasks for YOUR project

1. Copy the `server` folder, rename the project in `package.json`, point `.env` to **your** database.
2. Write ≥ 8 API routes for your domain: list with search/filter, detail, create, update, delete, one transaction (order/booking/transfer), two statistics routes (one SQL, one Mongo aggregation).
3. Build ≥ 4 pages: list, detail, a form that creates data, a dashboard. Change the colours/logo in `style.css` to your brand.
4. Validate input on the server (required fields, numbers > 0) and return clear error messages (400/404/409).
5. **Pairs:** add register/login with `bcrypt` password hashes and an admin page to add/edit items.

📦 **Deliverable 2 (end of Week 2)** – GitHub link + a 2-minute screen recording (Win + G or OBS) showing: the site running on localhost, a search, a detail page, creating a record (order/booking), the dashboard, and the new row visible in pgAdmin.

Week 2 grading checklist (30 points)

Criterion	Pts	What the teacher checks
Server runs	**4**	`npm install` + `npm start` work on the teacher's PC with a fresh .env
API routes	**8**	≥ 8 routes, correct HTTP methods and status codes, parameterised SQL everywhere
Uses both databases	**4**	At least one page combines PostgreSQL and MongoDB data
Transaction	**5**	BEGIN/COMMIT/ROLLBACK, stock/balance check, tested failure case
Front-end pages	**6**	≥ 4 pages, works without errors in the browser console, user text escaped
GitHub hygiene	**3**	Meaningful commits on ≥ 3 different days, no .env / node_modules in repo

3 Week 3 – Deploy, test & present

Goal: a public URL that works from any phone, a written report and a 7-minute live demo.

Day 11 (lab)	Day 12	Day 13 (lab)	Day 14	Day 15 (lab)
Supabase + Atlas: cloud databases	Deploy on Render, fix env variables	Test on phones, fix bugs, polish UI	Write report, make slides, record backup video	Final demo & defence

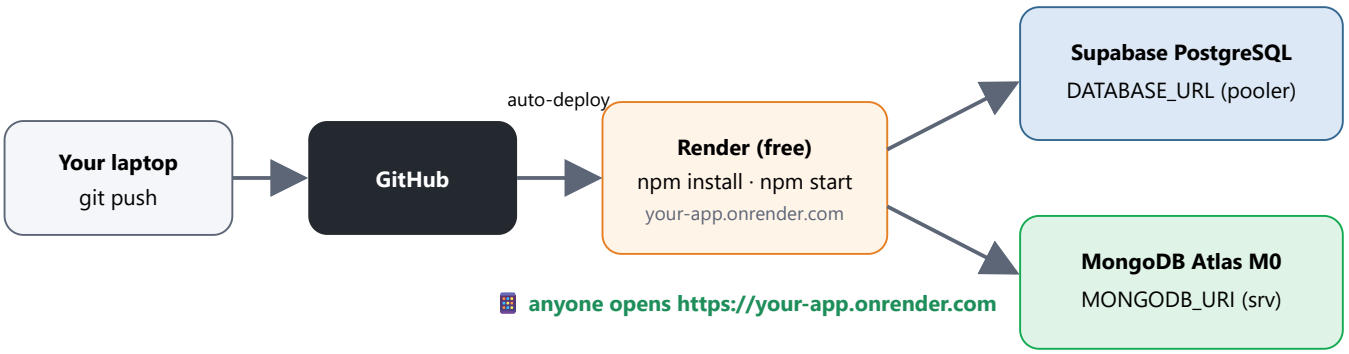


Figure 5.1 – Free production setup. Every `git push` redeploys the site automatically.

Step 1 · Cloud PostgreSQL on Supabase

1. Go to **supabase.com** → Start your project → sign in with GitHub.
2. New project : name `doritez`, choose a strong **database password** (write it down!), region **Central EU (Frankfurt)** – closest to Uzbekistan. Free plan. Wait ~2 minutes.
3. Left menu → SQL Editor → New query . Paste your `01_schema.sql`, click Run . Then paste and run `02_seed.sql`. Check in Table Editor that the tables have rows.
4. Click Connect (top bar) → tab Connection string → choose **Session pooler** (works on IPv4 networks such as Render). Copy the URI and replace `[YOUR-PASSWORD]`:

```
DATABASE_URL=postgresql://postgres.abcdxyz:YOUR_PASSWORD@aws-0-eu-central-
1.pooler.supabase.com:5432/postgres
PGSSL=true
```

> **Alternative: Neon**
> (neon.tech) – also free PostgreSQL. Create a project, open the SQL Editor , run your scripts, copy the connection string (it already contains `sslmode=require`), and set `PGSSL=true` .

Step 2 · Cloud MongoDB on Atlas

1. **mongodb.com/atlas** → Try Free → create a cluster → choose **M0 (Free)**, provider AWS, region Frankfurt.
2. Database Access → Add New Database User : username `student`, password without special symbols like `@ : / ?` (they break the URL).
3. Network Access → Add IP Address → Allow access from anywhere (`0.0.0.0/0`) – needed because Render's IP address changes.

4. **Database → Connect → Drivers** → copy the `mongodb+srv://…` string, insert your password.
5. Load your data: in Compass click **New connection**, paste the srv string, connect, then run your `mongo_setup.js` the same way as in Week 1 (VS Code playground or _MONGOSH).

Step 3 · Test locally with the cloud databases

Put the two cloud strings into your local `.env` (keep the lab version as a comment), restart `npm run dev` and open `/api/health`. If it works on your laptop with cloud databases, it will work on Render.

Step 4 · Deploy the website on Render

1. **render.com** → sign in with GitHub → **New +** → **Web Service** → select your repository.
2. Fill in the form:

Root Directory	`server` (the folder that contains package.json)
Runtime	Node
Build Command	`npm install`
Start Command	`npm start`
Instance Type	Free
Environment Variables	`DATABASE_URL`, `PGSSL = true`, `MONGODB_URI`, `MONGODB_DB` (do **not** set PORT – Render sets it)

3. Click **Deploy Web Service**. Watch the **Logs** tab until you see `SavdoShop running on …` and "Your service is live 🎉".
4. Open `https://YOUR-APP.onrender.com/api/health`, then the home page – on your phone too.

> **Free tier facts:**
> the service sleeps after 15 minutes without visitors; the first visit then takes ~50 seconds. **Open your site 2 minutes before your demo.** Supabase pauses a project after 7 days of no activity – open it the day before.

Plan B · Instant public link with Cloudflare quick tunnel

If deployment fails on demo day, you can still show your laptop's site through a public https link – no account needed:

```
winget install --id Cloudflare.cloudflared      # one time (or download cloudflared.exe from GitHub releases)
npm start                                        # terminal 1: your site on port 3000
cloudflared tunnel --url http://localhost:3000   # terminal 2
# ... Your quick Tunnel has been created! Visit it at:
# https://random-words-here.trycloudflare.com
```

The link works only while both terminals are open. Use it for demos, not as permanent hosting.

Step 5 · Final testing checklist

- [ ] Home page loads on a phone (mobile data, not lab Wi-Fi)
- [ ] Search and filter return correct results
- [ ] Detail page shows data from both databases
- [ ] Create/order/booking works; new row visible in Supabase
- [ ] Failure case shows a friendly error (out of stock, empty form)
- [ ] Dashboard shows numbers from aggregations
- [ ] No red errors in Chrome DevTools console (`F12`)
- [ ] No passwords in GitHub (search your repo for "password")
- [ ] README.md explains how to run the project
- [ ] Backup video recorded (in case Internet fails)

📦 **Deliverable 3 (end of Week 3)** – (1) public URL, (2) GitHub link, (3) report PDF (8–12 pages, template in Chapter 6), (4) slides (8–10), (5) 3-minute backup video. Upload all links in one text file `submission.txt`.

Week 3 grading checklist (40 points)

Criterion	Pts	What the teacher checks
Deployment	**10**	Public URL works during grading; cloud DBs used; secrets in environment variables
Testing & quality	**6**	Checklist above complete; error handling; looks good on a phone
Report	**10**	All sections of the template, ER diagram, query explanations, index experiment, screenshots
Live demo	**8**	Clear 7-minute demo following the script, works live
Defence (Q&A)	**6**	Student explains own code and SQL, answers 2–3 questions

6 · Report & presentation templates

6.1 Report structure (8–12 pages)

#	Section	Content	Pages
1	Title page	Project name, logo, student name(s), group, course, date, public URL + GitHub link	1
2	Business idea	Problem in Uzbekistan, target users, how the product earns money, 2 competitors	1
3	Architecture	Diagram like Figure 1.2; why PostgreSQL for X, MongoDB for Y	1
4	Database design	ER diagram, table descriptions, constraints, normalisation decisions	2
5	Queries & analytics	5 most interesting SQL + 3 Mongo queries with result screenshots and the business question each answers	2
6	Big data experiment	Rows generated, EXPLAIN ANALYZE before/after index, table of timings, explanation	1
7	Website & API	List of routes (method, URL, purpose), 4 screenshots, transaction explanation	1–2
8	Deployment & testing	Hosting choices, test checklist, problems you solved	1
9	Startup roadmap	Next features, how to scale, what data you would analyse with 1 million users	½

6.2 Seven-minute demo script

Time	What you show / say
0:00	**Problem & idea** (1 slide): "In Tashkent it takes 5 pharmacies to find one medicine. DoriTez shows stock in all of them."
0:45	**Architecture** (1 slide): browser → Express → PostgreSQL + MongoDB, hosted on Render/Supabase/Atlas.
1:30	**ER diagram**: point at the M:N relation and one CHECK constraint.
2:15	**Live site on a phone**: search, detail, create an order/booking.
3:45	**Proof in the database**: show the new row in Supabase Table Editor; show the failure case (out of stock → rollback).
4:45	**Big data**: dashboard + EXPLAIN ANALYZE before/after index (46 ms → 5 ms).
5:45	**One piece of code** you are proud of (e.g. the transaction).
6:30	**Startup roadmap** and thank you. Then questions.

Typical defence questions

– Why is `order_items` needed? · What happens if two people buy the last item? · Why NUMERIC and not FLOAT for money? · What does your index speed up, and what does it slow down? · Why are reviews in MongoDB and not in PostgreSQL? · How do you protect against SQL injection?

7 · Troubleshooting – common errors

#	Error / symptom	Cause	Fix
1	`'node' is not recognized as an internal or external command`	Node.js not installed or terminal opened before install	Install Node LTS from nodejs.org, then **close and reopen** the terminal / VS Code.
2	pgAdmin: `password authentication failed for user "postgres"`	Wrong password	Ask the lab admin for the password; at home it is the one you chose during installation.
3	pgAdmin: `connection refused … port 5432`	PostgreSQL service stopped	`Win` + `R` → `services.msc` → *postgresql-x64-17* → Start.
4	Compass: `connect ECONNREFUSED 127.0.0.1:27017`	MongoDB service stopped	`services.msc` → *MongoDB Server* → Start.
5	`npm install` fails with `UNABLE_TO_GET_ISSUER_CERT_LOCALLY` or `SELF_SIGNED_CERT_IN_CHAIN`	Antivirus (e.g. Avast/Kaspersky "HTTPS scanning") or university proxy intercepts HTTPS	Ask the admin to allow registry.npmjs.org, or temporarily turn off HTTPS scanning. Last resort (lab only): `npm config set strict-ssl false`, and set it back to true afterwards.
6	`ERROR: relation "products" does not exist`	Query Tool opened on the wrong database, or schema not run	Check the database name in the Query Tool tab; run `01_schema.sql` in *that* database.
7	`ERROR: insert or update on table "order_items" violates foreign key constraint`	Inserting a child row before its parent exists	Insert parents first (categories → products → orders → order_items), or check the id exists.
8	`duplicate key value violates unique constraint "customers_email_key"`	Same e-mail inserted twice	Use another e-mail, or `ON CONFLICT (email) DO NOTHING`. Rerun the whole schema script to reset.
9	`new row … violates check constraint "products_stock_check"`	Your rule works! Stock would go below 0	This is expected – handle it in the API and show a friendly message.
10	Server prints `Missing DATABASE_URL or MONGODB_URI`	No `.env` file, or it is named `.env.txt`	In VS Code create the file exactly `.env` in the `server` folder (same folder as package.json).
11	`Error: listen EADDRINUSE: address already in use :::3000`	Server already running in another terminal	Close the other terminal (`Ctrl` + `C`) or set `PORT=3001` in .env.
12	`Cannot use import statement outside a module`	`"type": "module"` missing in package.json	Use the provided package.json, or add `"type": "module"`.
13	Page shows but list is empty; console: `Failed to fetch` / 500	API error	Look at the **server terminal** – the real error is printed there. Open the API URL directly in the browser.
14	`Unexpected token '<' … is not valid JSON`	fetch() called a URL that returned an HTML page (404)	Check the route spelling: `/api/products` vs `/api/product`.
15	Prices appear as `"650000.00"` (string)	pg returns NUMERIC as text to avoid precision loss	Convert with `Number(p.price)` in JavaScript, or `::float` in SQL for statistics.

#	Error / symptom	Cause	Fix
16	Supabase from Render: `ENETUNREACH` / IPv6 address	Direct connection is IPv6-only	Use the **Session pooler** connection string (port 5432, host `…pooler.supabase.com`).
17	`self-signed certificate in certificate chain` (cloud Postgres)	SSL setting	Set `PGSSL=true`; db.js then uses `ssl: { rejectUnauthorized: false }`.
18	Atlas: `MongoServerSelectionError … timed out`	IP not allowed	Atlas → Network Access → add `0.0.0.0/0`. Lab Wi-Fi may block port 27017 – try a phone hotspot.
19	Atlas: `bad auth : authentication failed`	Wrong user/password, or special characters in password	Create a new DB user with a letters-and-digits password; paste it into the URI.
20	Render deploy: `Cannot find module …/package.json`	Root Directory wrong	Settings → Root Directory = `server`.
21	Render site loads forever the first time	Free instance was sleeping	Wait ~50 s. Open it before your demo.
22	Git: `remote origin already exists` / push rejected	Remote added twice, or GitHub has commits you don't	`git remote set-url origin URL`; then `git pull --rebase origin main` and push again.
23	MongoDB query returns nothing although data exists	Type mismatch: `"1"` (string) vs `1` (number)	Convert: `Number(req.params.id)` – exactly like in server.js.
24	EXPLAIN still shows Seq Scan after creating the index	Query returns a large part of the table, or statistics are old	Run `ANALYZE sales_log;`; filter on a selective value (one city + 30 days).

How to ask for help efficiently:

send the teacher (1) the exact error text (copy, not a photo), (2) the command or query you ran, (3) a screenshot of the file. 80% of problems are solved by reading the *first* red line carefully.

8 · Grading rubric (100 points)

Area	Pts	Excellent (full points)	Acceptable (half)	Weak (0–¼)
Database design (W1)	**15**	Normalised, ≥ 6 tables, M:N, all constraints, clear ERD	Works but missing constraints or M:N	Few tables, wrong keys, no ERD
SQL & MongoDB queries (W1)	**15**	20 SQL + 12 Mongo, all categories, explained, index experiment with numbers	Most queries, little explanation	Under half, copied from SavdoShop
API & transaction (W2)	**15**	≥ 8 routes, parameterised, validation, working rollback	Routes work, weak validation	Few routes, SQL injection possible
Website (W2)	**15**	≥ 4 pages, both DBs used, clean UI, no console errors	Pages work, basic look	Broken pages
Deployment (W3)	**10**	Public URL, cloud DBs, env variables	Only quick tunnel / partial	Not deployed
Report (W3)	**10**	Complete template, good screenshots, honest reflection	Missing 2–3 sections	Very short
Demo & defence (W3)	**14**	On time, works live, explains own code confidently	Some hesitation, small failures	Cannot explain the code
Code quality & GitHub	**6**	Readable, comments, regular commits, README	One big commit	No repo
Total 100		**Bonus (+5 max):** login with bcrypt (individuals), charts with Chart.js, partitioned table, a working Telegram bot that queries your API, real users testing your site.		

Academic honesty.
You may reuse the SavdoShop starter code and use AI assistants for help, but you must understand and be able to explain every line. Projects that are only SavdoShop with renamed words receive at most 40 points.

Appendix A · Cheat sheets

PostgreSQL

```
-- create / change structure
CREATE TABLE t (id SERIAL PRIMARY KEY, name TEXT NOT NULL);
ALTER TABLE t ADD COLUMN price NUMERIC(12,2) DEFAULT 0;
ALTER TABLE t ADD CONSTRAINT price_pos CHECK (price >= 0);
CREATE INDEX idx_t_name ON t(name);
DROP TABLE IF EXISTS t CASCADE;

-- CRUD
INSERT INTO t (name) VALUES ('a'), ('b') RETURNING id;
SELECT * FROM t WHERE name ILIKE '%a%' ORDER BY id LIMIT 10
OFFSET 0;
UPDATE t SET price = 100 WHERE id = 1;
DELETE FROM t WHERE id = 2;

-- grouping and joins
SELECT c.name, COUNT(*), SUM(p.price)
FROM products p JOIN categories c USING (category_id)
GROUP BY c.name HAVING COUNT(*) > 1;

-- useful functions
now(), date_trunc('month', ts), ts::date, COALESCE(x, 0),
ROUND(x, 2), LOWER(s), CONCAT(a, ' ', b),
generate_series(1, 10)

-- performance
EXPLAIN ANALYZE SELECT ...;
ANALYZE t;
```

MongoDB (mongosh)

```
use("mydb")                                  // switch
database
db.items.insertOne({ name: "a", tags: ["x"] })
db.items.insertMany([{...}, {...}])
db.items.find({ price: { $gt: 100 } })    // $gt $gte $lt
$lte $ne $in
db.items.find({}, { name: 1, _id: 0 }).sort({ price: -1
}).limit(5)
db.items.countDocuments({ city: "Tashkent" })
db.items.updateOne({ _id: id }, { $set: { price: 90 },
$inc: { views: 1 } })
db.items.updateMany({}, { $push: { tags: "new" } })
db.items.deleteMany({ price: { $lt: 0 } })

db.orders.aggregate([
  { $match: { status: "paid" } },
  { $group: { _id: "$city", total: { $sum: "$amount" },
              avg: { $avg: "$amount" } } },
  { $sort: { total: -1 } },
  { $lookup: { from: "cities", localField: "_id",
               foreignField: "name", as: "info" } },
  { $project: { _id: 0, city: "$_id", total: 1 } }
])

db.items.createIndex({ city: 1, ts: -1 })
db.items.find({...}).explain("executionStats")
```

Git in 6 commands

```
git status                      # what changed?
git add .                       # stage everything
git commit -m "Add booking API" # save a snapshot with a message
git push                        # upload to GitHub
git pull                        # download teammates' changes
git log --oneline               # history
```

HTTP status codes you will use

Code	Meaning	Use it when…
200	OK	Normal GET response
201	Created	POST created a new record
400	Bad Request	Missing/invalid input from the user
404	Not Found	No record with this id
409	Conflict	Business rule broken: out of stock, time slot already booked, insufficient balance
500	Server Error	A bug – read the server terminal

Appendix B · Full server source code

The complete, tested `server.js` of the reference project (also in `BigData_Starter_Code/server/src/`). Starter code folder contents: `sql/` (4 scripts), `mongo/` (2 scripts), `server/` (Express app, 6 front-end files).

server.js — JAVASCRIPT

```
// SavdoShop API + website  (Express, PostgreSQL, MongoDB)
import express from 'express';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import { pool, connectMongo, mongo } from './db.js';

const app = express();
const __dirname = path.dirname(fileURLToPath(import.meta.url));

app.use(express.json());                                  // read JSON bodies
app.use(express.static(path.join(__dirname, '..', 'public'))); // serve the website

// Small helper so every async route sends errors to the error handler
const wrap = (fn) => (req, res, next) => fn(req, res, next).catch(next);

// ---------- Placeholder product picture (works offline) ----------
// <img src="/placeholder.svg?text=Atlas+dress">  - replace with real photo URLs later
app.get('/placeholder.svg', (req, res) => {
  const text = String(req.query.text || 'Product').slice(0, 30).replace(/[<>&"']/g, '');
  const hue = [...text].reduce((h, ch) => (h * 31 + ch.charCodeAt(0)) % 360, 7);
  res.type('image/svg+xml').send(
    `<svg xmlns="http://www.w3.org/2000/svg" width="400" height="300">
       <rect width="400" height="300" fill="hsl(${hue},55%,85%)"/>
       <text x="200" y="160" font-family="Arial" font-size="30" text-anchor="middle"
             fill="hsl(${hue},45%,30%)">${text}</text></svg>`);
});

// ---------- Health check: is everything connected? ----------
app.get('/api/health', wrap(async (req, res) => {
  const pg = await pool.query('SELECT now() AS time');
  const reviews = await mongo().collection('reviews').estimatedDocumentCount();
  res.json({ ok: true, postgresTime: pg.rows[0].time, mongoReviews: reviews });
}));

// ---------- Categories (PostgreSQL) ----------
app.get('/api/categories', wrap(async (req, res) => {
  const { rows } = await pool.query('SELECT category_id, name FROM categories ORDER BY name');
  res.json(rows);
}));

// ---------- Product list with search, filter, pagination ----------
// GET /api/products?q=silk&category=1&page=1
app.get('/api/products', wrap(async (req, res) => {
  const q = req.query.q ? `%${req.query.q}%` : null;
  const category = req.query.category ? Number(req.query.category) : null;
  const page = Math.max(1, Number(req.query.page) || 1);
  const pageSize = 12;

  // $1, $2 ... are parameters: the driver escapes them (protects against SQL injection)
  const { rows } = await pool.query(
    `SELECT p.product_id, p.name, p.price, p.stock, p.image_url, c.name AS category
       FROM products p
       JOIN categories c ON c.category_id = p.category_id
      WHERE ($1::text IS NULL OR p.name ILIKE $1)
        AND ($2::int  IS NULL OR p.category_id = $2)
      ORDER BY p.product_id
      LIMIT $3 OFFSET $4`,
```

```
    [q, category, pageSize, (page - 1) * pageSize]
  );
  res.json(rows);
}));

// ---------- Product detail: PostgreSQL row + MongoDB reviews ----------
app.get('/api/products/:id', wrap(async (req, res) => {
  const id = Number(req.params.id);
  const { rows } = await pool.query(
    `SELECT p.*, c.name AS category FROM products p
       JOIN categories c ON c.category_id = p.category_id
      WHERE p.product_id = $1`, [id]);
  if (rows.length === 0) return res.status(404).json({ error: 'Product not found' });

  const reviews = await mongo().collection('reviews')
    .find({ productId: id }, { projection: { _id: 0 } })
    .sort({ createdAt: -1 }).limit(20).toArray();
  const avg = reviews.length
    ? reviews.reduce((s, r) => s + r.rating, 0) / reviews.length : null;

  // log a "view" event for analytics (do not wait for it)
  mongo().collection('events').insertOne({ type: 'view', productId: id, ts: new Date(), device: 'web' })
    .catch((e) => console.error('event log failed', e.message));

  res.json({ ...rows[0], reviews, avgRating: avg });
}));

// ---------- Add a review (MongoDB) ----------
app.post('/api/products/:id/reviews', wrap(async (req, res) => {
  const rating = Number(req.body.rating);
  const comment = String(req.body.comment || '').trim();
  if (!(rating >= 1 && rating <= 5) || comment.length < 3) {
    return res.status(400).json({ error: 'Rating 1-5 and a comment (min 3 letters) are required' });
  }
  const doc = { productId: Number(req.params.id), rating, comment,
                author: String(req.body.author || 'Guest'), createdAt: new Date() };
  await mongo().collection('reviews').insertOne(doc);
  res.status(201).json({ ok: true });
}));

// ---------- Place an order: one SQL transaction ----------
// body: { customer: {fullName, email, phone, city}, items: [{productId, quantity}], paymentMethod }
app.post('/api/orders', wrap(async (req, res) => {
  const { customer, items, paymentMethod = 'cash' } = req.body;
  if (!customer?.email || !customer?.fullName || !Array.isArray(items) || items.length === 0) {
    return res.status(400).json({ error: 'Customer name, email and at least one item are required' });
  }
  if (!['cash', 'card', 'payme', 'click'].includes(paymentMethod)) {
    return res.status(400).json({ error: 'Unknown payment method' });
  }

  const client = await pool.connect();   // all queries below use the SAME connection
  try {
    await client.query('BEGIN');

    const c = await client.query(
      `INSERT INTO customers (full_name, email, phone, city)
       VALUES ($1, $2, $3, COALESCE($4, 'Tashkent'))
       ON CONFLICT (email) DO UPDATE SET full_name = EXCLUDED.full_name
       RETURNING customer_id`,
      [customer.fullName, customer.email, customer.phone || null, customer.city || null]);
    const customerId = c.rows[0].customer_id;

    const o = await client.query(
      'INSERT INTO orders (customer_id) VALUES ($1) RETURNING order_id', [customerId]);
    const orderId = o.rows[0].order_id;
```

```
    let total = 0;
    for (const it of items) {
      const qty = Number(it.quantity);
      if (!Number.isInteger(qty) || qty < 1) throw Object.assign(new Error('Bad quantity'), { status: 400 });
      // decrease stock only if enough is left - atomic check
      const p = await client.query(
        `UPDATE products SET stock = stock - $1
          WHERE product_id = $2 AND stock >= $1
          RETURNING price, name`, [qty, it.productId]);
      if (p.rowCount === 0) {
        throw Object.assign(new Error(`Not enough stock for product ${it.productId}`), { status: 409 });
      }
      const price = Number(p.rows[0].price);
      total += price * qty;
      await client.query(
        'INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES ($1, $2, $3, $4)',
        [orderId, it.productId, qty, price]);
    }

    await client.query('UPDATE orders SET total = $1 WHERE order_id = $2', [total, orderId]);
    await client.query(
      'INSERT INTO payments (order_id, method, amount) VALUES ($1, $2, $3)',
      [orderId, paymentMethod, total]);

    await client.query('COMMIT');

    // analytics events go to MongoDB after the money part is safe
    mongo().collection('events').insertMany(items.map((it) => ({
      type: 'purchase', productId: Number(it.productId), customerId, city: customer.city || 'Tashkent',
      device: 'web', ts: new Date()
    }))).catch((e) => console.error('event log failed', e.message));

    res.status(201).json({ orderId, total });
  } catch (err) {
    await client.query('ROLLBACK');      // undo EVERYTHING from this order
    throw err;
  } finally {
    client.release();                    // give the connection back to the pool
  }
}));

// ---------- Dashboard statistics ----------
app.get('/api/stats/sales-by-city', wrap(async (req, res) => {
  const { rows } = await pool.query(
    `SELECT cu.city, COUNT(*)::int AS orders, SUM(o.total)::float AS revenue
       FROM orders o JOIN customers cu ON cu.customer_id = o.customer_id
      WHERE o.status <> 'cancelled'
      GROUP BY cu.city ORDER BY revenue DESC`);
  res.json(rows);
}));

app.get('/api/stats/top-viewed', wrap(async (req, res) => {
  const top = await mongo().collection('events').aggregate([
    { $match: { type: 'view' } },
    { $group: { _id: '$productId', views: { $sum: 1 } } },
    { $sort: { views: -1 } },
    { $limit: 5 }
  ]).toArray();
  // enrich with product names from PostgreSQL
  const ids = top.map((t) => t._id);
  const { rows } = await pool.query(
    'SELECT product_id, name FROM products WHERE product_id = ANY($1::int[])', [ids]);
  const names = Object.fromEntries(rows.map((r) => [r.product_id, r.name]));
  res.json(top.map((t) => ({ productId: t._id, name: names[t._id] || '?', views: t.views })));
}));

app.get('/api/stats/funnel', wrap(async (req, res) => {
```

```
  const rows = await mongo().collection('events').aggregate([
    { $group: { _id: '$type', count: { $sum: 1 } } }
  ]).toArray();
  res.json(Object.fromEntries(rows.map((r) => [r._id, r.count])));
}));

// ---------- Errors ----------
app.use('/api', (req, res) => res.status(404).json({ error: 'Unknown API route' }));
app.use((err, req, res, next) => {
  console.error(err);
  res.status(err.status || 500).json({ error: err.message || 'Server error' });
});

// ---------- Start ----------
const PORT = process.env.PORT || 3000;
connectMongo()
  .then(() => app.listen(PORT, () => console.log(`SavdoShop running on http://localhost:${PORT}`)))
  .catch((err) => { console.error('Could not connect to MongoDB:', err.message); process.exit(1); });
```

package.json JSON

```
{
  "name": "savdoshop-server",
  "version": "1.0.0",
  "description": "Big Data course project: PostgreSQL + MongoDB + Express website",
  "type": "module",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "node --watch src/server.js"
  },
  "engines": {
    "node": ">=20"
  },
  "dependencies": {
    "dotenv": "^16.4.5",
    "express": "^4.21.2",
    "mongodb": "^6.12.0",
    "pg": "^8.13.1"
  }
}
```

Guide version 1.0 · October 2026 · All code in this guide was executed and tested before publication (PostgreSQL 17 engine, MongoDB Community server, Node.js 22). Screenshots marked REAL were captured from the running reference project; images marked ILLUSTRATION are simplified drawings of third-party tools whose exact look may differ by version.